第 2-3 讲 • 冷库：数据库与迁移

> **模块 2 • 认清记账簿的骨架** ｜ 建议用时 **40 分钟**（阅读 22 + 动手 18）
> 前置要求：完成 2-1、2-2

本节目标

读完并做完这一讲，你能：

1. 说出**数据库**和"普通文件"的区别，以及为什么记账软件必须用它。
2. 看懂记账簿的**表结构**：一笔账在数据库里长什么样。
3. 解释一个关键设计：**为什么金额存的是"分"而不是"元"**，以及不这么做会出什么事故。
4. 理解**软删除**：为什么删掉的记录其实还在文件里。
5. 讲清**迁移**是什么，以及为什么"改数据库结构"是件必须谨慎的事。

一、为什么需要数据库

1.1 现实比喻：从流水账本到档案室

天真的做法：把所有账目写在一个文本文件里。

```
2026-09-28,支出,12.00,水果店
2026-09-28,支出,18.50,超市
2026-09-27,收入,12550.00,工资
...一万行...
```

问题立刻出现：

需求	文本文件的窘境
查 9 月一共花了多少	要把整个文件读一遍，一行行加
找"水果店"的消费	要全文搜索
同时记两笔账	两个人同时写，内容会互相覆盖
电脑突然断电	文件可能写到一半，变成半行乱码
账户改名了	得把所有记录里的旧名字全改一遍

数据库就是来解决这些问题的：

能力	数据库怎么做
快速查询	建**索引**，像书的目录，不用翻全文
条件筛选	一句 `WHERE` 语句搞定
并发安全	内建**事务**机制，要么全成功要么全失败
断电安全	写日志 + 原子提交，保证不出现"半笔账"
数据一致性	**外键**约束，防止出现"不存在的账户"

记账簿用的是 SQLite——一种**不需要安装服务器**的数据库，整个数据库就是一个文件（`daily-ledger.db`）。这对本地软件是最合适的选择。

1.2 SQLite 的一个特点：文件头能认出来

你已经在第 1-2 讲见过：它的前 16 字节固定是 `SQLite format 3`。**所以任何工具拿到这个文件，都能立刻认出"我是 SQLite 数据库"。**

二、一笔账在数据库里长什么样

2.1 表 = 表格

数据库用"表"来存数据，就像 Excel 的一张表：

`records` 表（记账簿真实的结构，节选）：

字段名	类型	含义
`id`	TEXT	**主键**，唯一标识（形如 `a1b2c3...`）
`date`	TEXT	业务日期（`2026-09-28`）
`amount`	INTEGER	**金额（单位：分）**
`type`	TEXT	类型：`income` / `expense` / `transfer`
`account_id`	TEXT	账户（指向 `accounts` 表）
`category_id`	TEXT	分类（指向 `categories` 表）
`note`	TEXT	备注
`tags`	TEXT	标签（存成 JSON 字符串）
`created_at`	INTEGER	创建时间（Unix 毫秒）
`deleted_at`	INTEGER	**删除时间（有值表示已删）**

三个值得注意的设计：

① 金额是 `INTEGER`（整数），单位是分

`12.00` 元在数据库里存的是 `1200`。详见下一节——这是个重要决定。

② `date` 用 `TEXT` 存，格式固定 `YYYY-MM-DD`

为什么不用日期类型？因为**排序和比较可以直接按字符串来**：`'2026-09-28' > '2026-09-27'` 这个比较结果是正确的。用固定格式的文本，简单又可靠。

③ `deleted_at` —— 删除不是真删

删掉的记录，`deleted_at` 会被写上时间戳，但**那一行还在**。这叫**软删除**。

三、两个关键设计

3.1 为什么金额存"分"（这是个经典事故）

你可能觉得：金额不就是 12 元吗，存 `12.00` 有什么问题？

问题在浮点数。计算机用二进制表示小数，**有些十进制小数在二进制里是无限循环的**，只能存近似值。以下是**实测结果**（你可以自己跑）：

```
0.1 + 0.2        // 0.30000000000000004  ← 不是 0.3!
0.1 + 0.2 === 0.3   // false


1.1 * 3          // 3.3000000000000003
1.1 * 3 === 3.3     // false
```

最要命的是"元转分"这一步——记账软件天天要做：

```
8.7 * 100        // 869.9999999999999   ← 本该是 870!
1.005 * 100      // 100.49999999999999  ← 本该是 100.5!
```

这意味着什么：用户输入 `8.7 元`，程序算出 **869.9999999999999 分**，再一取整就变成 **869 分 = 8.69 元。用户记了 8.7 元，账本上却变成 8.69 元，凭空少了 1 分钱。**

更糟的是累积：

```
let s = 0
for (let i = 0; i < 10; i++) s += 0.1   // 0.9999999999999999 ← 不是 1!
```

在记账软件里，这意味着：用户记了很多笔账，每笔都可能带上一点点误差，加起来对不上账。**用户会认为软件算错了**——而这确实是算错了。

解决方案：**用整数存最小的货币单位。**

用户看到	数据库里存的
12.00 元	`1200`（分）
0.01 元	`1`（分）
12550.00 元	`1255000`（分）

整数的加减乘除永远精确，不存在误差。需要显示给人看时，再除以 100 转回元。

验证一下（对照组）：

```
// 浮点：累加 0.1 十次
let s = 0; for (let i = 0; i < 10; i++) s += 0.1     // 0.9999999999999999 ❌

// 整数：累加 10 分十次
let c = 0; for (let i = 0; i < 10; i++) c += 10       // 100 ✅ 正好是 1.00 元
```

这是记账类软件的行业标准做法。 你自己做任何涉及钱的软件，都要记住这条。

> **一个附带的好处**：数据库里还有一句 `CHECK (amount >= 0)` —— 意思是"金额不允许是负数"。方向由 `type` 字段决定（收入/支出/转账），金额本身永远是正数。**这样就不会出现"负的支出"这种让人困惑的数据。**

3.2 为什么要"软删除"

用户以为的删除：记录没了。 **实际的删除**：`deleted_at` 字段被写上时间戳，记录**还在文件里。**

为什么不真删：

理由	说明
误删可恢复	记账簿有「回收站」页，删掉的记录在那里，可以还原
撤销功能	删除后右下角弹出「撤销」，就是靠软删除实现的
数据可追溯	以后想分析"我删过哪些记录"，数据还在

代价：文件会变大（删掉的东西还占空间）。但对一个记账软件来说，这点空间远不如"误删能救回来"重要。

查询时怎么办：所有正常查询都会带上 `WHERE deleted_at IS NULL`（只要没被删的）。这就是为什么"删除"之后列表里看不到它，但它其实还在。

四、迁移：数据库结构的"版本管理"

4.1 问题：结构改了，老用户的数据怎么办

假设记账簿 1.0 版的 `records` 表**没有** `tags` 字段，2.0 版加上了。那么：

- **新用户**：建库时就是新结构，没问题。
- **老用户**：他的数据库文件是**旧结构**。程序按新结构去读 `tags`，会报错"没有这个字段"。

总不能让他删库重来吧——那他的账就没了。

解决办法：**迁移（Migration）**——一段小代码，负责"把旧结构改成新结构，同时保留所有数据"。

4.2 记账簿的真实迁移

项目里有三个迁移文件（这是真实的目录列表）：

```
src\main\db\migrations\
├─ 001_init.ts                建表（初始结构）
├─ 002_account_archive.ts     给账户加"归档"功能
├─ 003_category_archive.ts    给分类加"归档"功能
└─ index.ts                   迁移的执行器
```

每个迁移干一件事，编号递增。

执行器（`index.ts`）的机制：

```
const from = db.pragma('user_version', { simple: true })   // 当前版本号
const pending = migrations.filter((m) => m.version > from) // 找出还没跑的
for (const migration of pending) {
  const apply = db.transaction(() => {
    migration.up(db)                                        // 执行结构变更
    db.pragma(`user_version = ${migration.version}`)        // 更新版本号
  })
  apply()
}
```

翻译成人话：

> "看看数据库现在是第几版（ `user_version` ）→ 找出所有比它新的迁移 → 逐个执行 → 每执行完一个就把版本号加一。"

三个关键设计：

设计	为什么
用版本号记录进度	程序不用猜"改过没有"，读一个数字就知道
每个迁移包在事务里	要么整个成功、要么整个回滚，**不会改一半**
顺序追加，只升不降	已经发布过的迁移不能再改内容（用户的库可能已经执行过了）

第 3 条是一条铁律：如果你改了 `001_init.ts` 的内容，那么**已经装过老版本的用户永远不会执行这个改动**（因为他们的版本号已经 ≥ 1）。所以**改动只能靠新增迁移**，不能改旧的。

> **这条规则对你以后很重要**：如果你要给记账簿加一个字段，正确做法是**新建** `004_xxx.ts` ，而不是去改 `001_init.ts` 。

4.3 迁移出错了怎么办（真实场景）

项目里有个专门的验证脚本 `npm run verify:upgrade` ：它会**造一个旧版本的数据库，跑真实迁移，检查数据是否完好。**

为什么需要它：迁移是**唯一会动到用户已有数据的操作。**改错了，用户的账就毁了。所以必须有一个自动化的"升级演练"。

它安全吗？——我实测验证过：

检查项	结果
运行前你的数据库字节数	135168
运行结果	**21/21 通过**
运行后你的数据库字节数	**135168**（没变）
数据库文件修改时间	**没变**（说明根本没被写过）

结论：它完全在临时沙箱里跑，**不会碰你的真实数据**。你可以放心执行。

这也是模块 5 要讲的核心理念：危险的操作必须有自动化验证兜底。

五、动手任务

步骤 1：确认数据库文件的身份

```
cd "C:\Users\34123\Desktop\记账簿"
Format-Hex -Path "<你的数据目录>\daily-ledger.db" -Count 16
```

预期看到：`53 51 4C 69 74 65 20 66 6F 72 6D 61 74 20 33 00`，右侧注释 `SQLite format 3`。

这就是"SQLite 数据库"的身份证。

步骤 2：看一眼表结构是怎么写的

```
Get-Content src\main\db\migrations\001_init.ts | Select-String "CREATE TABLE IF NOT EXISTS
```

预期看到 7 张表：`accounts`、`categories`、`records`、`attachments`、`recurring`、`budgets`、`settings`。

思考：这 7 张表分别存什么？

表	存什么
`accounts`	账户（微信、支付宝……）
`categories`	分类（餐饮、交通……），**自引用**（父分类指向自己这张表）
`records`	**账目**（核心表）
`attachments`	收据图片
`recurring`	周期记账规则
`budgets`	预算
`settings`	设置（键值对）

这就是整个软件的数据全貌——7 张表装下了一个记账软件。

步骤 3：找到"金额是分"的铁证

```
Get-Content src\main\db\migrations\001_init.ts | Select-String "amount"
```

预期看到：`amount INTEGER NOT NULL CHECK (amount >= 0),`

三个信息一次读到：

1. `amount` 是 **INTEGER**（整数）→ 所以存的是分，不是元
2. `NOT NULL` → 金额不能为空
3. `CHECK (amount >= 0)` → **数据库层面**禁止负数

第 3 点很有意思：即使程序写错了，想插一笔负数金额，**数据库会直接拒绝**。这叫"约束"——**把规则写在数据层，而不是只靠代码自觉。**

步骤 4：验证软删除

```
Get-Content src\main\db\migrations\001_init.ts | Select-String "deleted_at"
```

预期看到：`deleted_at INTEGER`（在几张表里都出现）。

现在做个实验（在你的真实数据上，但**安全**）：

1. 打开记账簿，**删掉一笔记录**（随便选一笔不重要的）。
2. 记下删除前后 `daily-ledger.db` 的字节数：

```
(Get-Item "<你的数据目录>\daily-ledger.db").Length
```

1. **观察**：文件大小**没有明显变小**。

为什么：因为记录没被真正删掉，只是打了个"删除标记"（`deleted_at` 写了时间戳）。**数据还在文件里。**

1. 去「回收站」页，你会看到刚才那笔记录——**可以恢复**。
2. 恢复它，验证数据完好。

这一课的意义：你亲手验证了"软删除"的存在。**也验证了记账簿的删除是安全的**——删错了能救回来。

步骤 5：看迁移的执行逻辑

```
Get-Content src\main\db\migrations\index.ts | Select-String "user_version|transaction|pend
```

预期看到：

- `db.pragma('user_version', ...)` —— 读当前版本
- `migrations.filter((m) => m.version > from)` —— 找出待执行的
- `db.transaction(...)` —— 包在事务里

思考：为什么每个迁移要包在事务里？

答案：**为了防止"改一半"**。如果迁移执行到中间失败了（断电、程序崩溃），事务会**整体回滚**，数据库回到执行前的状态。否则可能变成"表建好了但数据没搬"的残缺状态——**那比没执行更糟。**

卡住了怎么办

现象	原因	怎么办
`Format-Hex` 不认识	PowerShell 版本较老	用 `Get-Content -Encoding Byte -TotalCount 16` 代替
删了记录后文件变小了	SQLite 可能做了回收	正常，重点看"回收站里还能找到它"
不敢删任何记录做实验	完全可以理解	**那就别删**，直接去回收站看看有没有历史删除记录即可
`verify:upgrade` 想跑一下	好习惯	`npm run verify:upgrade`（它用临时目录，**不碰你的数据**）

六、自测题

1. 为什么记账软件一定要用数据库，不能直接写文本文件？说出至少三个理由。

2. 12.00 元的金额，在记账簿的数据库里存的是什么？为什么要这样设计？

3. 你删掉一笔记录后，它真的从文件里消失了吗？怎么证明？

4. 迁移是什么？为什么不能直接去改 `001_init.ts` 来加字段？

5. `CHECK (amount >= 0)` 这样的约束有什么意义？

答案与解析

1. ① **查询快**（索引）；② **并发安全**（事务，不会互相覆盖）；③ **断电安全**（原子提交，不会出现半笔账）；④ **数据一致性**（外键，防止出现无效的账户引用）。 → 任答三个即可。核心是：**文本文件在"多、快、稳、准"四个方面都不够用。**

2. 存 `1200`（单位：分）。 → 因为计算机用二进制表示小数会有**精度误差**（`0.1 + 0.2 !== 0.3`）。记账软件里这种误差累积起来会导致"对不上账"。**用整数存最小货币单位是行业标准做法。**

3. 没有真正消失，只是 `deleted_at` 被写了时间戳（软删除）。 → 证明方式：① 文件大小没有明显减小；② **回收站里还能找到它并恢复。**

4. 迁移是**一段把旧数据库结构升级到新结构的代码**，保证老用户的数据不丢。 → 不能改 `001_init.ts`，因为**已经装过老版本的用户版本号已经 ≥ 1，永远不会再执行它。**正确做法是**新增一个** `004_xxx.ts` 。

5. 它是**写在数据层的规则**：即使程序代码写错了想插一笔负数金额，**数据库会直接拒绝。** → 这叫"约束"——**不依赖代码自觉，而是让数据层守住底线。**防御要有多层。

七、术语表

术语	一句话解释
数据库 / Database	专门用来高效、安全地存取结构化数据的软件
SQLite	不需要服务器的数据库，整个库就是一个文件
表 / Table	存一类数据的地方，类似 Excel 的一张表
字段 / Field	表的一列，如 `amount` 、 `date`
主键 / Primary Key	唯一标识一行的字段，这里是 `id`
外键 / Foreign Key	指向另一张表的字段，用来保证引用有效
索引 / Index	加速查询的结构，像书的目录
事务 / Transaction	一组操作"要么全成功、要么全回滚"
约束 / Constraint	数据层强制的规则，如"金额不能为负"
软删除 / Soft Delete	不真删，只标记删除时间，可恢复
迁移 / Migration	把旧数据库结构升级到新结构的代码
`user_version`	SQLite 里记录"我的结构是第几版"的数字
回滚 / Rollback	撤销本次事务中的全部改动
浮点数误差	二进制无法精确表示某些十进制小数导致的计算偏差
整数存分	用最小货币单位（分）的整数存金额，避免精度问题

八、本讲与项目的对应

- 你查看的文件：`src\main\db\migrations\001_init.ts`（建表）、`index.ts`（迁移执行器）
- 你做的实验：删一笔记录 → 观察到文件没变小 → 在回收站恢复
- **本讲会修改你的真实数据（删一笔记录）**，但**完全可恢复**——如果你不想动数据，跳过步骤 4 的实验即可
- 上一讲：2-2 传菜口：preload 与 IPC ｜ 下一讲：**2-4 装修队：React 与界面**

讲义版本：v1 ｜ 对应项目版本：0.1.0